multibyte_composition: recovering four key bytes from raw traces

What this folder shows

Section 5.1 quotes a result from the long version: using the full raw ASCAD traces (100,000 points each, instead of the usual 700-point window), four key bytes are recovered exactly from a few hundred traces. Four scripts, run in order:

1. `fetch_raw_ascad.py` downloads the ANSSI archive (4.4 GB), checks it, unpacks the 6 GB raw trace file, and checks that too. If the file is already there, it is only checked.
2. `scan_raw_ascad_pois.py` finds the points in the trace that relate to each key byte.
3. `train_raw_ascad_byte_attacker.py --target-byte B` trains a small network for one key byte on those points and saves its guesses on the attack traces. The paper trains bytes 0, 1, 2, 3, 4, 5, 8 and 15.
4. `compose_four_byte_recovery.py` picks the four best bytes and counts, for each number of traces, how often all four are recovered at once.

Level D checks the result.

Before you start

The Docker image (or the Python environment with `requirements-ml.txt`), 11 GB of free disk while unpacking, and the 4.4 GB download.

Run

```
python -m multibyte_composition.fetch_raw_ascad
```

```
$ python -m multibyte_composition.fetch_raw_ascad
datasets/raw/ascad_raw/ATMega8515_raw_traces.h5: checksum verified (raw traces)
[exit code 0, 1 min 9 s]
```

```
python -m multibyte_composition.scan_raw_ascad_pois
```

```
$ python -m multibyte_composition.scan_raw_ascad_pois
Scanning datasets/raw/ascad_raw/ATMega8515_raw_traces.h5
samples=20000 trace_len=100000 label_columns=48 key=[77, 251, 224, 242, 114, 33, 254, 16, 167, 141, 74, 220,
     142, 73, 4, 105]
processed columns 0:2048
processed columns 20480:22528
processed columns 40960:43008
processed columns 61440:63488
processed columns 81920:83968
processed columns 98304:100000
Done. Wrote results/multibyte_composition/poi_scan
[exit code 0, 57 s]
```

```
python -m multibyte_composition.train_raw_ascad_byte_attacker --target-byte 0
```

```
$ python -m multibyte_composition.train_raw_ascad_byte_attacker --target-byte 0
byte=0 device=cpu
raw=datasets/raw/ascad_raw/ATMega8515_raw_traces.h5 n_total=60000 trace_len=100000 base_pois=24 expanded_fea
    tures=374
{'epoch': 1, 'train_loss': 4.511685210800171, 'train_acc': 0.04795, 'val_loss': 3.589524269104004, 'val_acc'
    : 0.15440000593662262}
{'epoch': 2, 'train_loss': 2.8372331146240235, 'train_acc': 0.270425, 'val_loss': 2.2349112033843994, 'val_a
    cc': 0.38339999318122864}
{'epoch': 3, 'train_loss': 2.019215158081055, 'train_acc': 0.420875, 'val_loss': 1.8306177854537964, 'val_ac
    c': 0.4537999927997589}
{'epoch': 4, 'train_loss': 1.7122486515045166, 'train_acc': 0.481275, 'val_loss': 1.6293666362762451, 'val_a
    cc': 0.5004000067710876}
{'epoch': 5, 'train_loss': 1.5571004913330078, 'train_acc': 0.508025, 'val_loss': 1.4723453521728516, 'val_a
    cc': 0.5289000272750854}
{'epoch': 6, 'train_loss': 1.4567091352462769, 'train_acc': 0.5325, 'val_loss': 1.5397754907608032, 'val_acc
    ': 0.5081999897956848}
{'epoch': 7, 'train_loss': 1.3469295066833495, 'train_acc': 0.563575, 'val_loss': 1.358241319656372, 'val_ac
    c': 0.5548999905586243}
{'epoch': 8, 'train_loss': 1.307603094291687, 'train_acc': 0.572275, 'val_loss': 1.3890875577926636, 'val_ac
    c': 0.5436000227928162}
{'epoch': 9, 'train_loss': 1.2561079355239868, 'train_acc': 0.589375, 'val_loss': 1.3004268407821655, 'val_a
    cc': 0.5709999799728394}
{'epoch': 10, 'train_loss': 1.2292320028305053, 'train_acc': 0.59225, 'val_loss': 1.2228084802627563, 'val_a
    cc': 0.5927000045776367}
[... 5 lines omitted ...]
{'epoch': 16, 'train_loss': 1.04424747505188, 'train_acc': 0.647225, 'val_loss': 1.1877027750015259, 'val_ac
    c': 0.5957000255584717}
{'epoch': 17, 'train_loss': 1.0474901607513427, 'train_acc': 0.6441, 'val_loss': 1.1784374713897705, 'val_ac
    c': 0.6046000123023987}
{'epoch': 18, 'train_loss': 1.00211676902771, 'train_acc': 0.65775, 'val_loss': 1.1766127347946167, 'val_acc
    ': 0.6054999828338623}
{'epoch': 19, 'train_loss': 0.9960979377746582, 'train_acc': 0.657825, 'val_loss': 1.2442864179611206, 'val_
    acc': 0.5889000296592712}
{'epoch': 20, 'train_loss': 0.9701184679031372, 'train_acc': 0.669625, 'val_loss': 1.1272298097610474, 'val_
    acc': 0.619700014591217}
{'epoch': 21, 'train_loss': 0.9445877880096436, 'train_acc': 0.67735, 'val_loss': 1.157904863357544, 'val_ac
    c': 0.6129000186920166}
{'epoch': 22, 'train_loss': 0.9228891040802002, 'train_acc': 0.683, 'val_loss': 1.1509817838668823, 'val_acc
    ': 0.6104000210762024}
{'epoch': 23, 'train_loss': 0.9183218925476074, 'train_acc': 0.6858, 'val_loss': 1.0924911499023438, 'val_ac
    c': 0.6347000002861023}
{'epoch': 24, 'train_loss': 0.8758448049545288, 'train_acc': 0.699175, 'val_loss': 1.1887245178222656, 'val_
    acc': 0.6013000011444092}
{'epoch': 25, 'train_loss': 0.866223159790039, 'train_acc': 0.702175, 'val_loss': 1.1498908996582031, 'val_a
    cc': 0.6208000183105469}
{'epoch': 26, 'train_loss': 0.8551014095306396, 'train_acc': 0.703575, 'val_loss': 1.2124271392822266, 'val_
    acc': 0.6031000018119812}
{'epoch': 27, 'train_loss': 0.8731923820495605, 'train_acc': 0.698075, 'val_loss': 1.1844329833984375, 'val_
    acc': 0.602400004863739}
{'epoch': 28, 'train_loss': 0.838342523765564, 'train_acc': 0.7094, 'val_loss': 1.1774266958236694, 'val_acc
    ': 0.6141999959945679}
{'epoch': 29, 'train_loss': 0.8085810266494751, 'train_acc': 0.722675, 'val_loss': 1.1669788360595703, 'val_
    acc': 0.6110000014305115}
{'epoch': 30, 'train_loss': 0.7972917036056518, 'train_acc': 0.724975, 'val_loss': 1.2197093963623047, 'val_
    acc': 0.5958999991416931}
{'epoch': 31, 'train_loss': 0.7945099649429321, 'train_acc': 0.72765, 'val_loss': 1.178092122077942, 'val_ac
    c': 0.6105999946594238}
early stopping at epoch 31
{'target_byte': 0, 'correct_key': 77, 'device': 'cpu', 'features': 374, 'base_pois': 24, 'val_top1': 0.6347,
     'attack_top1': 0.6261, 'val_rank_at_5000': 0, 'val_top_key_at_5000': 77, 'attack_rank_at_50 …
[exit code 0, 1 min 57 s]
```

```
for b in 1 2 3 4 5 8 15; do python -m multibyte_composition.train_raw_ascad_byte_attacker --target-byte $b; done
```

```
$ for b in 1 2 3 4 5 8 15; do python -m multibyte_composition.train_raw_ascad_byte_attacker --target-byte $b
    ; done
[... earlier output omitted; key lines kept ...]
[...]
{'epoch': 4, 'train_loss': 5.526374110412598, 'train_acc': 0.007125, 'val_loss': 5.616174697875977, 'val_acc
    ': 0.0034000000450760126}
{'epoch': 5, 'train_loss': 5.495504537963868, 'train_acc': 0.00895, 'val_loss': 5.616761684417725, 'val_acc'
    : 0.003800000064074993}
{'epoch': 6, 'train_loss': 5.456371211242676, 'train_acc': 0.012175, 'val_loss': 5.633718013763428, 'val_acc
    ': 0.0020000000949949026}
{'epoch': 7, 'train_loss': 5.414058724975586, 'train_acc': 0.015075, 'val_loss': 5.644437313079834, 'val_acc
    ': 0.003599999938160181}
{'epoch': 8, 'train_loss': 5.368957651519775, 'train_acc': 0.0183, 'val_loss': 5.6615214347839355, 'val_acc'
    : 0.0038999998942017555}
{'epoch': 9, 'train_loss': 5.31204873046875, 'train_acc': 0.023525, 'val_loss': 5.683440685272217, 'val_acc'
    : 0.0040000000189989805}
{'epoch': 10, 'train_loss': 5.2614487968444825, 'train_acc': 0.0281, 'val_loss': 5.716784954071045, 'val_acc
    ': 0.0032999999821186066}
early stopping at epoch 10
{'target_byte': 8, 'correct_key': 167, 'device': 'cpu', 'features': 513, 'base_pois': 24, 'val_top1': 0.0051
    , 'attack_top1': 0.0043, 'val_rank_at_5000': 230, 'val_top_key_at_5000': 211, 'attack_rank_a …
byte=15 device=cpu
raw=datasets/raw/ascad_raw/ATMega8515_raw_traces.h5 n_total=60000 trace_len=100000 base_pois=24 expanded_fea
    tures=467
{'epoch': 1, 'train_loss': 5.616971869659424, 'train_acc': 0.004175, 'val_loss': 5.598978519439697, 'val_acc
    ': 0.004800000227987766}
{'epoch': 2, 'train_loss': 5.574897273254394, 'train_acc': 0.004675, 'val_loss': 5.593192100524902, 'val_acc
    ': 0.0037000000001117587}
{'epoch': 3, 'train_loss': 5.549068795776368, 'train_acc': 0.0063, 'val_loss': 5.5904154777526855, 'val_acc'
    : 0.0049999999888241291}
{'epoch': 4, 'train_loss': 5.510014268493652, 'train_acc': 0.008425, 'val_loss': 5.586073875427246, 'val_acc
    ': 0.00430000014603138}
{'epoch': 5, 'train_loss': 5.464591016387939, 'train_acc': 0.01025, 'val_loss': 5.585899829864502, 'val_acc'
    : 0.0049000000058114529}
{'epoch': 6, 'train_loss': 5.417757390594482, 'train_acc': 0.013025, 'val_loss': 5.5845417976379395, 'val_ac
    c': 0.0052000000014156103}
{'epoch': 7, 'train_loss': 5.36630477142334, 'train_acc': 0.016525, 'val_loss': 5.601146697998047, 'val_acc'
    : 0.0049000000058114529}
{'epoch': 8, 'train_loss': 5.315443386077881, 'train_acc': 0.0203, 'val_loss': 5.606040000915527, 'val_acc':
     0.004800000227987766}
{'epoch': 9, 'train_loss': 5.260475843811035, 'train_acc': 0.0251, 'val_loss': 5.614712715148926, 'val_acc':
     0.0065000000134110451}
{'epoch': 10, 'train_loss': 5.204805307006836, 'train_acc': 0.0282, 'val_loss': 5.6346540451049805, 'val_acc
    ': 0.0054000000140070915}
{'epoch': 11, 'train_loss': 5.1384163757324215, 'train_acc': 0.0352, 'val_loss': 5.68211030960083, 'val_acc'
    : 0.0049999999888241291}
{'epoch': 12, 'train_loss': 5.0756769973754885, 'train_acc': 0.040225, 'val_loss': 5.6839799880981445, 'val_
    acc': 0.0057999999926239252}
{'epoch': 13, 'train_loss': 5.005690699768066, 'train_acc': 0.046775, 'val_loss': 5.718158721923828, 'val_ac
    c': 0.0059000000222027302}
{'epoch': 14, 'train_loss': 4.925694054412841, 'train_acc': 0.056475, 'val_loss': 5.763463497161865, 'val_ac
    c': 0.00559999980032444}
{'epoch': 15, 'train_loss': 4.845280448150635, 'train_acc': 0.06405, 'val_loss': 5.7883830070495605, 'val_ac
    c': 0.0046000000102072954}
{'epoch': 16, 'train_loss': 4.7587927658081055, 'train_acc': 0.07315, 'val_loss': 5.88934326171875, 'val_acc
    ': 0.00559999980032444}
{'epoch': 17, 'train_loss': 4.679345816040039, 'train_acc': 0.08345, 'val_loss': 5.912873268127441, 'val_acc
    ': 0.0049999999888241291}
early stopping at epoch 17
{'target_byte': 15, 'correct_key': 105, 'device': 'cpu', 'features': 467, 'base_pois': 24, 'val_top1': 0.006
    5, 'attack_top1': 0.0044, 'val_rank_at_5000': 0, 'val_top_key_at_5000': 105, 'attack_rank_at …
[exit code 0, 11 min 40 s]
```

```
python -m multibyte_composition.compose_four_byte_recovery --campaigns 250 \
  --trace-counts 1,2,5,10,20,50,100,200,500,1000,2000
```

```
$ python -m multibyte_composition.compose_four_byte_recovery --campaigns 250 --trace-counts 1,2,5,10,20,50,1
    00,200,500,1000,2000
```

```
Selected bytes: [0, 1, 5, 15]
{'trace_count': 1, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 0, 'empirical_full_succ
    ess': 0.0, 'full_kl_binom_upper': 0.01191141971501136, 'dc_product_empirical_byte_success':  …
{'trace_count': 2, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 0, 'empirical_full_succ
    ess': 0.0, 'full_kl_binom_upper': 0.01191141971501136, 'dc_product_empirical_byte_success':  …
{'trace_count': 5, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 0, 'empirical_full_succ
    ess': 0.0, 'full_kl_binom_upper': 0.01191141971501136, 'dc_product_empirical_byte_success':  …
{'trace_count': 10, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 0, 'empirical_full_suc
    cess': 0.0, 'full_kl_binom_upper': 0.01191141971501136, 'dc_product_empirical_byte_success': …
{'trace_count': 20, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 4, 'empirical_full_suc
    cess': 0.016, 'full_kl_binom_upper': 0.04364222571943082, 'dc_product_empirical_byte_success …
{'trace_count': 50, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 10, 'empirical_full_su
    ccess': 0.04, 'full_kl_binom_upper': 0.077855192727511, 'dc_product_empirical_byte_success': …
{'trace_count': 100, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 35, 'empirical_full_s
    uccess': 0.14, 'full_kl_binom_upper': 0.19925414076946063, 'dc_product_empirical_byte_succes …
{'trace_count': 200, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 104, 'empirical_full_
    success': 0.416, 'full_kl_binom_upper': 0.49318781737017653, 'dc_product_empirical_byte_succ …
{'trace_count': 500, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 224, 'empirical_full_
    success': 0.896, 'full_kl_binom_upper': 0.9369162083759379, 'dc_product_empirical_byte_succe …
{'trace_count': 1000, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 250, 'empirical_full
    _success': 1.0, 'full_kl_binom_upper': 1.0, 'dc_product_empirical_byte_success': 1.0, 'dc_pr …
{'trace_count': 2000, 'campaigns': 250, 'selected_bytes': '0,1,5,15', 'full_successes': 250, 'empirical_full
    _success': 1.0, 'full_kl_binom_upper': 1.0, 'dc_product_empirical_byte_success': 1.0, 'dc_pr …
[exit code 0, 2 min 42 s]
```

```
$ python -c "import pandas as pd; print(pd.read_csv('results/multibyte_composition/byte_attackers/multibyte_
    composition_summary.csv')[['trace_count', 'campaigns', 'full_successes', 'empirical_full_success', 'full
    _kl_binom_upper', 'median_full_rank']].to_string(index=False))"
 trace_count  campaigns  full_successes  empirical_full_success  full_kl_binom_upper  median_full_rank
           1        250               0                   0.000             0.011911           99776.5
           2        250               0                   0.000             0.011911           12731.0
           5        250               0                   0.000             0.011911            5689.5
          10        250               0                   0.000             0.011911            1713.0
          20        250               4                   0.016             0.043642             247.5
          50        250              10                   0.040             0.077855              32.0
         100        250              35                   0.140             0.199254               9.0
         200        250             104                   0.416             0.493188               1.0
         500        250             224                   0.896             0.936916               0.0
        1000        250             250                   1.000             1.000000               0.0
        2000        250             250                   1.000             1.000000               0.0
[exit code 0, 1 s]
```

What to look for

- Each training run prints how well the network does on held-out traces. Some bytes do not learn well on a CPU; they are simply not picked. The runs behind the paper and the reruns all pick bytes 0, 1, 5 and 15.
- In the summary, `full_successes` is the number of tries (out of 250) in which all four bytes were right with that many traces. `median_rank` should be 0 from 500 traces on.
- The exact counts change from one training run to the next. The table in the paper came from one such run (40, 94, 191, 206 and 221 out of 250 at 100 to 2,000 traces); the rerun on the Mac gives 35, 104, 224, 250 and 250. The claim that the check tests holds every time: four bytes recovered from 500 traces on.